

Carmen ↔ OCR — Email Automation Integration Contract

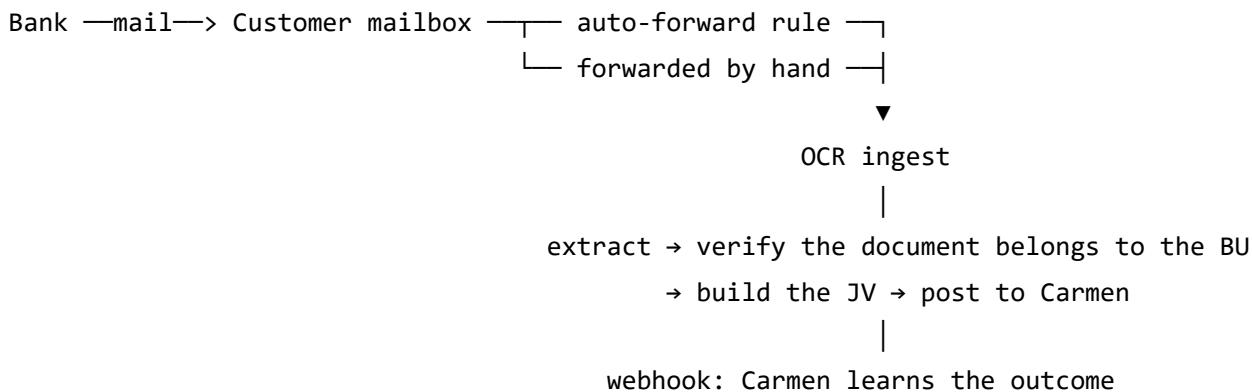
Status: DRAFT — for review by the Carmen team.

Sections marked **OPEN** need a decision from Carmen before we can finalise the implementation. Everything else is settled on our side and will not change without notice here.

Companion documents: [EMAIL_FLOW.md](#) (the v1 pilot design, now superseded), [Security_Trust_Overview.md](#).

0. What this is

Banks email commission / fee reports to hotels. Today a user downloads the file and uploads it into the OCR wizard by hand. In this version the customer sets up **one auto-forward rule** in their mailbox and everything after that is automatic:



The three things that make this different from the pilot:

1. **No human approval step.** Documents post to Carmen automatically.
2. **Settings live in Carmen.** The customer configures **everything on Carmen's screens**; Carmen calls our API to store it. The OCR app has no settings UI for this feature.
3. **Carmen is notified by webhook**, so it can react without polling us.

What the **customer does** (the whole setup)

1. Turn on Email Automation in Carmen (**only available with an active monthly package**).
2. Copy the forwarding address Carmen shows them, and either set an auto-forward rule in their mailbox, or simply forward the bank's mail to that address by hand whenever they receive one. Both work, and they can mix the two.

That is the entire customer-facing setup. Everything else — tax ID, GL mapping, bank rules — comes from data Carmen already holds or the customer has already configured in the OCR app.

0.1 The two arrival modes

Both are supported, but they carry different amounts of information and the system compensates differently. Carmen does not need to configure anything per mode — this section exists so the behaviour is not surprising.

	Auto-forward	Manual forward
Sender of the mail we receive	the bank	the employee who forwarded it
Which BU it belongs to	the address it was sent to	the address it was sent to
Which bank issued it	matched from the sender address	detected from the document itself
Bank's DKIM signature	survives, and we check it	broken by the forward — cannot be checked
What proves the document belongs to this BU	the tax ID on the document	the tax ID on the document

Two consequences worth stating plainly:

- **The tax ID check is what makes manual forward safe.** A manually forwarded mail carries no verifiable proof that a bank produced the attachment, so the document's own content is the only trustworthy evidence. This is the main reason §2.4 is mandatory.
- **A document forwarded twice is not charged twice.** If the same report arrives both automatically and by hand, the second one is recognised by document number and rejected as a duplicate, with the credit refunded.

1. Identity

A **tenant** in the OCR system is the pair (host, bu) :

Field	Meaning	Example
host	hostname of the customer's Carmen instance	hotelgroup.carmenwork.com
bu	business-unit code, lowercased	hq , bkk01

This is the same pair `POST /api/v1/auth/exchange` already resolves today, so a BU that has ever logged into the OCR app already exists on our side. Every API call and every webhook in this document carries `host + bu` ; there is no separate tenant id for Carmen to store (we return ours for logging convenience, but Carmen never needs to send it back).

Naming convention: our JSON is **snake_case** . Carmen's existing API is `PascalCase` . We follow **each side's own convention rather than mixing.**

2. Part A — Settings API (Carmen → OCR)

2.1 Authentication — **OPEN (needs Carmen's answer)**

Two workable options; we can implement either, and the choice mostly depends on whether Carmen calls us from the browser (a user is present) or server-to-server.

Option A — API key + request signature (server-to-server).

Carmen holds a key we issue **per Carmen host**. Every request carries the key plus an HMAC-SHA256 signature of the exact request body, and a timestamp to stop replay.

`PUT /api/v1/carmen/settings`

Authorization: `ApiKey ocr_live_7f3a91...`

X-OCR-Timestamp: `1785000000`

X-OCR-Signature: `sha256=9c1f...`

Content-Type: `application/json`

112 279

Pros: works with no user session, survives session expiry, easy to automate.

Cons: Carmen must store a secret and rotate it.

Option B — the user's Carmen token (reuse the existing SSO exchange).

Carmen's settings screen sends the logged-in user's Carmen token to

POST /api/v1/auth/exchange (an endpoint that already exists and that we already validate against Carmen's own API), then calls the settings endpoint with the resulting OCR session JWT.

Pros: no new credential anywhere, and every settings change is attributable to a real Carmen user (better audit trail). *Cons:* only works while a user is on screen.

Our recommendation: Option B for the settings screen, and Option A later if Carmen ever needs to push settings without a user (bulk provisioning, migrations). Starting with B costs nothing and can be extended to A without changing the payload.

2.2 Read current settings

GET /api/v1/carmen/settings?host=hotelgroup.carmenwork.com&bu=hq

why not??

GET /api/v1/carmen/settings/{host}/{bu}

```

{
  "host": "hotelgroup.carmenwork.com",
  "bu": "hq",
  "enabled": true,
  "entitled": true,           // active monthly package – see §3.3
  "ingest_address": "...",   // OPEN – see §2.5
  "tax_ids": ["0105536000123"],
  "rules": [
    {
      "id": "b3f1...",
      "bank_code": "KTC",           // null = "Other" (generic prompt + auto-detect)
      "bank_label": null,          // display name, used when bank_code is null
      "bank_sender_email": "no-reply@ktc.co.th",
      "filename_pattern": "MDR",   // case-insensitive substring; null = any file
      "has_password": true,        // never the password itself
      "is_active": true
    }
  ],
  "status": {
    "documents_total": 128,
    "last_received_at": "2026-07-30T09:12:00Z",
    "ready": true,                // false = something below blocks ingestion
    "blockers": []               // e.g. ["no_tax_id", "not_entitled", "no_rule"]
  }
}

```

enabled function

bank active

2.3 Write settings

PUT /api/v1/carmen/settings

```

{
  "host": "hotelgroup.carmenwork.com",
  "bu": "hq",
  "enabled": true,
  "tax_ids": ["0105536000123"],           // REQUIRED — see §2.4
  "rules": [
    {
      "bank_code": "KTC",                  // the rule's identity — required
      "bank_sender_email": "no-reply@ktc.co.th", // optional hint, see below
      "filename_pattern": "MDR",
      "pdf_password": "1234",              // write-only: omit = keep, "" = clear
      "is_active": true
    }
  ]
}

```

- The payload **replaces** the BU's rule list (send the **full list, not a delta**).
- A rule is **keyed** by **bank_code**, not by the sender address. Manually forwarded mail arrives from an employee, so the issuing bank is detected from the document itself; the **sender address is only a fast path for automatically forwarded mail**. `bank_sender_email` may therefore be `null`, and a BU that only ever forwards by hand never has to fill it in.
- `pdf_password` is accepted on write, **stored encrypted**, and **never returned by any endpoint**. Reads expose `has_password: true/false` only. When the issuing bank is not yet known at the moment a protected file is opened, we try the passwords of that BU's active rules — they are all the customer's own, and there are only a handful.
- Supported `bank_code` values today: BBL , KBANK , SCB , BAY , KTC , GHL , PAYPAL , SIAMPAY , or `null` for anything else.

Errors are returned as 422 with a per-field list so Carmen can render them inline:

```

{ "errors": [ { "field": "tax_ids[0]", "code": "invalid_checksum",
               "message": "Tax ID checksum does not match" } ] }

```

2.4 Tax IDs — required, and why

The BU's 13-digit tax ID must be set before Email Automation can be switched on.

It is the control that stops a document belonging to another legal entity from being posted into this BU's books — the one failure that automatic posting cannot recover

from, and that no email-level check can detect (a correctly-routed mail can still carry the wrong company's invoice, e.g. when a customer forwards the wrong message).

Every document we extract is checked against this list before it is posted. A document whose tax IDs do not include the BU's is never posted; the credit is refunded and Carmen is notified.

- Send **all** tax IDs that legitimately appear on this BU's bank documents (several branches or legal entities under one BU is fine — it is a list).
- Format: 13 digits, no dashes or spaces. We validate the standard check digit and reject a malformed value at write time rather than silently at 3am.
- A tax ID may belong to **only one BU across the whole system**. A second BU claiming the same number is rejected with 409 — please surface that error to the user, it usually means a copy-paste mistake.
- **We would like Carmen to fill this automatically from its company master** rather than asking the user to type it. It is data Carmen already holds, and a typed number is a number that can be typed wrong.

Request to the Carmen team: confirm which field in Carmen's company/BU master holds the tax ID (and branch, if separate), so we can agree the mapping.

2.5 The forwarding address — OPEN (tied to a decision on our side)

Where the customer forwards their bank mail depends on a transport decision we are still making. Either way Carmen's screen only has to **display a string we return** in `ingest_address` and tell the user to forward to it — the field exists in the contract now so Carmen's UI does not change later.

Model	What <code>ingest_address</code> contains
Per-BU alias (<i>preferred</i>)	a unique address we generate, e.g. <code>ingest-7f3a91@...</code> — nothing to type, nothing to verify, and one BU cannot receive another's mail
Shared mailbox	one address shared by all customers; the BU is identified from the original recipient header, so the customer's own receiving address must also be registered

Supporting manual forwarding effectively decides this. A mail forwarded by hand arrives from an employee's own address and carries no trace of who originally received it, so the

address it was *sent to* is the only thing left that identifies the BU. With a shared mailbox we would have to register every employee who might forward something — exactly the kind of setup step this design is trying to remove.

We will confirm before implementation. **No action needed from Carmen** beyond displaying the returned value.

3. Part B — Webhooks (OCR → Carmen)

3.1 Envelope, signing and delivery

Every event is a `POST` with the same envelope:

```
{
  "id": "evt_01J8...",           // unique per event – use it to dedupe
  "type": "subscription.activated",
  "occurred_at": "2026-07-31T03:15:00Z",  // UTC, ISO-8601
  "tenant": { "host": "hotelgroup.carmenwork.com", "bu": "hq" },
  "data": { }                     // per-event, see below
}
```

Headers:

Header	Meaning
X-OCR-Event-Id	same value as <code>id</code> — dedupe key
X-OCR-Event-Type	same value as <code>type</code> — lets Carmen route before parsing
X-OCR-Timestamp	unix seconds, included in the signed string
X-OCR-Signature	<code>sha256=<hex></code> — HMAC-SHA256 of <code>"{timestamp}.{raw_body}"</code> using the shared secret

Verification on Carmen's side: recompute the HMAC over the *raw* body bytes (before JSON parsing) and compare in constant time; reject if `X-OCR-Timestamp` is more than 5 minutes from now.

Delivery semantics:

- **At-least-once.** Retries mean Carmen can receive the same event twice — dedupe on `id` , and make handlers idempotent.
- Success = any `2xx` , returned within **10 seconds**. Anything else is a retry.
- Retries use exponential backoff over roughly 24 hours; after that the event is parked as undeliverable and raises an alert on our side. We can replay parked events on request.
- Ordering is **not** guaranteed. Use `occurred_at` when order matters (e.g. an `activated` arriving after a `lapsed` for the same BU).
- One endpoint URL per Carmen host is enough; tell us if you would rather have one URL per event type.

3.2 notification.created

Fires when a new in-app notification is created for a BU, so Carmen can show the badge without polling.

```
{
  "type": "notification.created",
  "data": {
    "notification_id": "a91c...",
    "notification_type": "order_paid",      // see the list below
    "unread_count": 3,
    "created_at": "2026-07-31T03:15:00Z"
  }
}
```

The payload deliberately carries **no message text** — copy differs per language and changes often. Carmen should render its own text from `notification_type` , or call our notifications endpoint to fetch the full list.

Current `notification_type` values: `order_created` , `order_paid` , `order_rejected` , `order_expired` , `credits_low` , plus the document events introduced by this feature (§3.4). We will add to this list over time — **treat an unknown type as "something happened", not as an error.**

3.3 subscription.activated / subscription.lapsed

This is what tells Carmen whether Email Automation may be switched on for a BU.

```
{
  "type": "subscription.activated",
  "data": {
    "plan_code": "growth",
    "billing_period": "monthly",          // "monthly" | "annual"
    "doc_allowance": 500,                 // documents per month
    "period_start": "2026-07-31T00:00:00Z",
    "period_end": "2026-08-30T23:59:59Z",
    "entitlements": { "email_automation": true }
  }
}
```

```
{
  "type": "subscription.lapsed",
  "data": {
    "plan_code": "growth",
    "ended_at": "2026-08-30T23:59:59Z",
    "entitlements": { "email_automation": false }
  }
}
```

- `activated` fires when a package purchase is approved, and again on each renewal.
- **`lapsed` matters as much as `activated`** — without it Carmen would leave the setting switched on after the package ends. It fires from the same daily job that closes an expired subscription window on our side.
- `entitlements` is an object on purpose: more features will appear there. Read the key you care about and ignore the rest.
- The `entitled` field in `GET /settings` (§2.2) is always authoritative — if Carmen ever misses an event, re-reading settings gives the current truth. Please treat the webhook as an optimisation, not as the only source.

3.4 `document.posted` / `document.failed` — **proposed**

Not in the original request, but with no human approval step these are the only way Carmen (or the customer) learns what happened to a forwarded document.

```
{
  "type": "document.posted",
  "data": {
    "document_id": "d41f...",
    "bank_code": "KTC",
    "doc_no": "INV-2026-0001",
    "doc_date": "2026-07-30",
    "total_amount": "12345.67",
    "jv_reference": "...",          // shape depends on §4
    "posted_at": "2026-07-31T03:16:00Z"
  }
}
```

```
{
  "type": "document.failed",
  "data": {
    "document_id": "d41f...",
    "reason_code": "tax_id_mismatch",
    "message": "This document belongs to a different company.",
    "credit_refunded": true
  }
}
```

reason_code values (stable identifiers; the message is display text and may change):

tax_id_mismatch , bank_not_identified , mapping_incomplete , unbalanced_jv ,
duplicate_document , unreadable_document , wrong_pdf_password , out_of_credits ,
carmen_rejected .

Question for the Carmen team: do you want these, and if so should they also raise an in-app notification for the customer, or only feed Carmen's own screens?

3.5 What we need to start sending

1. **Endpoint URL** per Carmen host (a staging URL first).
2. **A shared secret per endpoint**, exchanged out-of-band — please do not send it over email or chat; we will agree a channel.
3. Confirmation that the receiver responds 2xx **before** doing its own work (accept-then-process), so a slow downstream job does not turn into a retry storm.

4. Part C — Posting the JV — **OPEN (the main decision)**

Every other part of this document is independent of this choice, but this one changes which side owns the posting code, so we would like to settle it first.

The problem: posting a JV to Carmen today uses the **logged-in user's Carmen token**, which lives for about 30 minutes. Automatic posting happens on a schedule with nobody logged in, so a session token cannot be used.

Option 1 — Carmen pulls and posts (no credential leaves Carmen).

We send `document.ready` ; Carmen fetches the prepared JV from us and posts it internally.

```
GET /api/v1/carmen/documents/{document_id}    → the JV payload, ready to post
PATCH /api/v1/carmen/documents/{document_id} → { "jv_no": "..." } (tell us the outcome)
```

Pros: we never hold a Carmen credential — the smallest possible attack surface, and nothing to rotate. *Cons:* the duplicate guard and the posting retry live on Carmen's side.

Option 2 — Carmen issues a long-lived service token per BU.

Sent to us with the settings payload; we store it encrypted and post directly.

Pros: no change to Carmen's posting code; our side stays in control end to end.

Cons: we hold a long-lived customer credential, which needs rotation, revocation and an expiry alarm.

Option 3 — Client-credentials exchange.

Carmen exposes a token endpoint; we exchange a client id/secret for a short-lived access token per posting run.

Pros: the standard answer, and the best security/ownership balance.

Cons: Carmen has to build the endpoint if it does not exist yet.

Our preference: Option 3 if the endpoint can be built, otherwise Option 1. We would rather not hold long-lived customer credentials (Option 2) unless the other two are impossible.

Whichever is chosen, the JV content itself is unchanged from what the wizard posts today (`JvhSeq/JvhDate/Prefix/JvhSource/Detail[]`), and the GL accounts come from the mapping the customer has already configured in the OCR app.

5. Checklist for the Carmen team

#	We need	Blocks
1	Decision on §4 — how the JV gets posted	the whole posting path
2	Decision on §2.1 — API key or user token	the settings endpoint
3	Webhook endpoint URL + secret exchange channel (§3.5)	both webhooks
4	Which Carmen field holds the BU tax ID (§2.4)	switching the feature on
5	Yes/no on the proposed document.* events (§3.4)	outcome reporting
6	Confirmation that Email Automation is gated on the monthly package only	entitlement logic

6. Out of scope for v1

AP-invoice ingestion by email · per-tenant mailboxes · customer-editable GL mapping from Carmen's side (it stays in the OCR app) · posting anything other than credit-card commission / fee documents.